Academic Year: 2026-2027
Semester: III
Class / Branch: SE IT
Subject: ADSAL
Name of Instructor: Ms. Sneha Dalvi

Name of Student:Nishant Suryarao
Student ID:26204001
Date of Performance:22/09/2026
Date of Submission:22/09/2026

Experiment No.5 Aim:

Implement Prim's and Kruskal's algorithms.

Code:

1. Prim's Algorithm Program

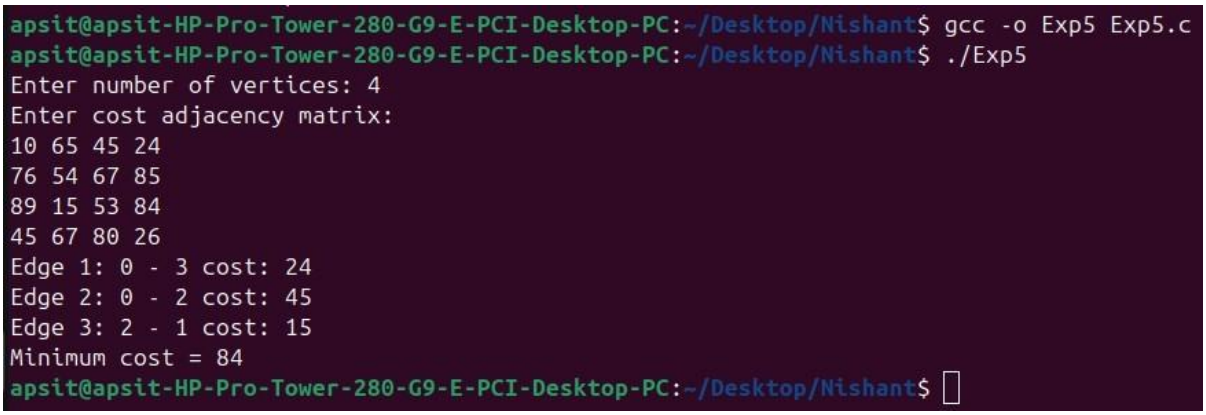
```
#include <stdio.h> #define INF 999999
int main() {  int n, i, j, a, b, edges = 0,
total = 0, min;  int cost[10][10],
visited[10] = {0};


 printf("Enter number of vertices: ");
scanf("%d", &n);  printf("Enter cost
adjacency matrix:\n");  for (i = 0; i < n;
i++)  for (j = 0; j < n; j++) {  scanf("%d",
&cost[i][j]);  if (cost[i][j] == 0) cost[i][j] =
INF;
 }


 visited[0] = 1;  while
(edges < n - 1) {  min =
INF;
 for (i = 0; i < n; i++)  if (visited[i])  for (j = 0; j < n; j++)  if
(!visited[j] && cost[i][j] < min) {  min = cost[i][j];  a = i; b = j;
```

```
} printf("Edge %d: %d - %d cost: %d\n", edges + 1, a, b,
min); visited[b] = 1;total += min; edges++; }
printf("Minimum cost = %d\n", total); return 0;
}
```

Output Screen Shots:



A. Prim's Algorithm

Definition: Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.

Algorithm:

1. Start with the given weighted graph.
2. Select any vertex as the starting vertex.
3. Mark the selected vertex as visited.
4. Find the minimum-weight edge connecting a visited vertex to an unvisited vertex.
5. Add the edge to the MST.
6. Mark the new vertex as visited.
7. Repeat until \(V-1\) edges are selected.
8. Display the MST and its total cost.
9. Stop.

Conclusion: Thus, Prim's algorithm was successfully implemented to find the Minimum Spanning Tree with minimum total cost.

2. Kruskal's Algorithm Program

```
#include <stdio.h>
#define INF 999
int parent[10]; int
find(int i) {  while
(parent[i]) i =
parent[i];return i;
}
int uni(int i, int j) {
if (i != j) {
parent[j] = i;
return 1;  }  return
0;
}
int main() {  int cost[10][10], n, i, j, ne = 1, min, u, v, a, b,
total = 0;  printf("Enter number of vertices: ");
scanf("%d", &n);  printf("Enter cost adjacency matrix
(999 for no edge):\n");  for (i = 0; i < n; i++)  for (j = 0; j <
n; j++)  scanf("%d", &cost[i][j]);  while (ne < n) {  min =
INF;
 for (i = 0; i < n; i++)  for (j = 0; j < n; j++)  if (cost[i][j] <
min) {  min = cost[i][j];  a = u = i;  b = v = j;  }  u = find(u);
v = find(v);if (uni(u, v)) {  printf("Edge %d: (%d %d) cost:
%d\n", ne++, a, b, min);  total += min;
 }  cost[a][b] = cost[b][a] =
INF;
```

```
} printf("Total cost: %d\n",
total); return 0;
}
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o Exp5 Exp5.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./Exp5
Enter number of vertices: 4
Enter cost adjacency matrix (999 for no edge):
65 85 76 54
56 76 09 45
34 24 14 54
98 76 08 35
Edge 1: (3 2) cost: 8
Edge 2: (1 2) cost: 9
Edge 3: (2 0) cost: 34
Total cost: 51
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion: Prim's and Kruskal's algorithms were successfully implemented to find the Minimum Spanning Tree (MST) of a weighted graph. Both algorithms prod

B. Kruskal's Algorithm

Definition: Kruskal's algorithm is a greedy algorithm that constructs a Minimum Spanning Tree by selecting edges in increasing order of weight while avoiding cycles.

Algorithm:

1. Start with the given weighted graph.
2. Sort all edges in increasing order of weight.
3. Select the minimum-weight edge.
4. Check whether adding the edge creates a cycle.
5. If no cycle is formed, add the edge to the MST.
6. Otherwise, discard the edge.
7. Repeat until \(V-1\) edges are selected.
8. Display the MST and its total cost.
9. Stop.

Conclusion: Thus, Kruskal's algorithm was successfully implemented to construct a Minimum Spanning Tree while avoiding cycles.

Complexity to remember:

- Prim's: \(O(V^2)\) for an adjacency-matrix implementation.
- Kruskal's: Typically \(O(E\log E)\) with edge sorting and an efficient disjoint-set structure.